

학습_2-3단계_실행가이드

학습 2·3단계 실행 가이드

1단계(v2 추출기 + 12채널)까지는 이미 완료. 남은 두 번을 돌리는 방법과 결과 읽는 법입니다.

스크립트: `본격/모터_학습_v3.py`

`모터_학습_v3.py`

0. 왜 두 번 더 돌리나

한 번에 하나씩만 바뀌야 원인을 가릴 수 있기 때문입니다.

지금까지 온 경로:

단계	추출기	채널	누수방지 단위	상태
3차 발표	구버전	12	<code>group_key</code>	완료 (0.83~0.87)
1단계	v2 (LUT+rownorm)	12	<code>group_key</code>	완료
2단계	v2	15 (rpm 추가)	<code>group_key</code>	← 이번에
3단계	v2	15	<code>timestamp</code>	← 이번에

각 화살표가 딱 하나씩만 다릅니다.

3차 발표 → 1단계 추출기 교체 효과
1단계 → 2단계 rpm 효과 ← 이번 작업의 핵심 질문
2단계 → 3단계 zsplit 누수 크기

한꺼번에 바꿔놓고 비교하면 "성능이 변했는데 뭐 때문인지 모름" 상태가 됩니다.

1. 실행 방법

사전 확인 (한 번만)

```
pip install -U sktime pyarrow
```

리허설 — 5분, 꼭 먼저

```
cd C:\Users\jaesung\Desktop\매너졸프
```

```
python 모터_학습_v3.py ^  
--data "C:\Users\jaesung\Desktop\매너졸프\특징데이터" ^
```

```
--out "C:\Users\jaesung\Desktop\매너졸프\모델" ^
--group-by group_key --limit 50
```

출력에서 3가지만 확인:

```
15채널 융합(early fusion): 3센서 x 5피쳐      ← 15채널 맞는지
차종 3개                                     ← 81개 같은 숫자면 실패
  IONIQ   파일   xxx   클래스 ['DEMAG', 'ECC10', 'ECC20', 'NORMAL', 'REDUC']
```

⚠️ “차종 3개”가 아니라 수십 개로 나오면 메타 정보를 못 읽은 것입니다.

특징데이터 폴더 옆에 샘플인덱스.parquet 이 있는지 확인하세요.

2단계 — rpm 효과 측정 (약 1시간 20분)

```
python 모터_학습_v3.py --data "C:\Users\jaesung\Desktop\매너졸프\특징데이
터" --out "C:\Users\jaesung\Desktop\매너졸프\모델" --group-by group_key
```

→ 학습리포트_fuse_rpm_gk.txt

3단계 — 누수 단위 강화 (약 1시간 20분)

```
python 모터_학습_v3.py --data "C:\Users\jaesung\Desktop\매너졸프\특징데이
터" --out "C:\Users\jaesung\Desktop\매너졸프\모델"
```

→ 학습리포트_fuse_rpm.txt

- -group-by 를 안 쓰면 기본값이 timestamp 입니다. 그 차이 하나뿐입니다.

💡 파일명이 모드별로 달라서 덮어쓰기 걱정은 없습니다.

밤에 두 줄 이어서 걸어두셔도 됩니다.

2. 용어 — 이번에 새로 나온 것

group_key vs timestamp (누수방지 단위)

같은 주행에서 나온 데이터가 훈련용과 시험용으로 갈리면 점수가 부풀려집니다. 그걸 막는 단위입니다.

```
group_key = 22_08_12_11_12_18_01_zsplit001_Current_U
            22_08_12_11_12_18_01_zsplit002_Current_U  ← 다른 group_key
            └── timestamp 는 같음 ───┘
```

zsplit 은 한 주행 안에서 잘라낸 구간 번호입니다. 즉 위 둘은 같은 주행의 연속된 구간입니다. rpm을 보면 명 확합니다:

```
zsplit001   rpm 1935 → 2341
zsplit002   rpm 2343 → 3272 → 4222      ← 2341에서 이어짐
```

단위	의미
group_key	zspllit 단위. 3차 발표에서 쓴 것. 같은 주행의 다른 구간이 train/test로 갈릴 수 있음
timestamp	주행 전체. 한 주행이 통째로 한쪽에만 들어감. 더 보수적

3차 발표에서 누수 방지를 안 한 게 아닙니다. 단위가 조금 느슨했던 것이고, 3단계는 그걸 더 조입니다.

-limit

파일 수를 제한해서 빠르게 돌려보는 리허설용입니다. 결과는 못 씁니다.

3. 결과 읽는 법

리포트 맨 위에 어떤 조건으로 돌렸는지 찍힙니다. 먼저 이것 확인하세요.

설정: fuse_rpm_gk 누수방지 그룹 = group_key (3차 발표와 동일 단위)

[학습 요약]

slot	chunks	그룹수	macro_F1	acc	rpm단독F1
IONIQ·FUSED15	xxxxx	xxx	0.xxx±0.xxx	0.xxx	0.xxx

- macro_F1 — 주 지표. ± 뒤는 폴드 간 편차 (클수록 불안정)
- rpm단독F1 — 아래 [진단 2]에서 설명
- 그룹수 — 몇 개 단위로 나눠서 검증했는지

[진단 1] 클래스당 그룹 수

IONIQ · FUSED15 DEMAG:xx ECC10:xx ECC20:xx NORMAL:xx REDUC:xx

값	해석
10 이상	통과. 성능 수치를 믿어도 됨
2~9	주의. 폴드 편차(±)가 크면 특히
1	CV 자체가 생략됨. macro-F1이 아예 안 나옴

2단계와 3단계의 이 숫자를 비교해보세요. group_key 가 timestamp 보다 큼니다(한 주행이 여러 개로 쪼개지니까). 몇 배 차이인지가 곧 zspllit이 얼마나 잘게 쪼개고 있었는지입니다.

[진단 2] rpm 지름길 — 2·3단계의 핵심

NIRO · FUSED15 rpm단독 0.xxx / 전체 0.xxx (설명비율 xx%)

rpm 값 하나만 넣고 학습한 결과입니다. 다른 특징은 전부 뺐습니다.

rpm단독 F1	판정
0.60 이상	● 위험
0.40~0.60	● 주의
0.40 미만	● 통과

왜 높으면 나쁜가:

샘플데이터에서 NIRO는 rpm만으로 100% 분류됐습니다. 자석이 열화돼서 rpm이 높은 게 아니라, **DEMAG 측정을 그날 빠르게 달렸을 뿐**이었습니다(클래스당 주행 1회).

이런 모델을 실제 차에 올리면:

- 고속도로에서 멀쩡한 차가 5000rpm → **DEMAG 오경보**
- 저속에서 진짜 자석이 열화 → **놓침**

도로에서 rpm은 고장이 아니라 **운전자가 정합니다**.

📖 이런 걸 **지름길 학습(shortcut learning)**이라고 합니다.

늑대와 허스키를 구별하는 모델이 실제로는 **배경의 눈**을 보고 있던 유명한 사례가 있습니다.

클래스별 rpm 범위도 같이 찍힙니다:

DEMAG	2645 ~ 2983	
ECC10	3360 ~ 4068	← DEMAG와 안 겹침

겹치지 않으면 rpm이 그대로 정답표가 됩니다. 본데이터에서 서로 겹치면 안심해도 됩니다.

[진단 3] DEMAG-NORMAL — 프로젝트 본 과제

IONIQ · FUSED15 DEMAG recall 0.xxx D→N xxx N→D xxx

- **DEMAG recall** — 진짜 DEMAG 중 맞힌 비율. 높을수록 좋음
- **D→N** — DEMAG를 NORMAL로 잘못 본 건수. 낮을수록 좋음
- **N→D** — 반대 방향

3차 발표 기준 **D→N이 873건**이었습니다. 이것과 비교하세요.

4. 최종 비교표 — 이렇게 채우면 됩니다

조건	추출기	채널	그룹	macro-F1	D→N
3차 발표	구	12	group_key	0.83~0.87	873
1단계 (완료)	v2	12	group_key	?	?
2단계	v2	15	group_key	?	?
3단계	v2	15	timestamp	?	?

해석 시나리오

① 1단계 → 2단계에서 올랐다 + rpm단독 F1이 낮다

→  rpm이 제대로 된 보조 정보로 쓰였습니다. 채택.

② 올랐는데 rpm단독 F1도 높다 (0.6+)

→  지름길일 가능성. 오른 게 실력이 아니라 정답을 본 것일 수 있습니다.

1단계와 성능이 비슷하면 rpm을 빼는 게 낫습니다.

③ 거의 안 올랐다

→ rpm은 없어도 됩니다. 채널 3개 아끼는 게 이득입니다.

다만 차수 정규화(rpm으로 주파수축을 보정하는 방식)는 아직 안 써봤으니,
rpm 자체가 무용지물이라는 뜻은 아닙니다.

④ 2단계 → 3단계에서 떨어졌다

→ zsplit 누수가 있었다는 뜻입니다. 성능이 나빠진 게 아니라 원래 그 정도였던 것입니다.

발표에서는 “누수 가능성을 더 보수적으로 차단하고 재측정”이라고 설명하면 됩니다.

⑤ 2단계 ≈ 3단계

→  3차 발표 수치에 누수가 없었다는 증거입니다. 좋은 결과입니다.

5. 발표에서 나올 만한 질문

Q. rpm 넣은 건 반칙 아닌가? 회전속도만 보고 맞힌 거 아닌가?

→ [진단 2] 결과를 보여주면 됩니다. rpm 하나만으로 학습했을 때 F1이 얼마인지 측정해줬습니다.
낮으면 “rpm은 보조 정보로만 쓰였다”고 답할 수 있습니다.

Q. 3차 발표보다 성능이 떨어졌는데?

→ 측정 기준이 더 엄격해졌습니다. 누수 방지 단위를 zsplit에서 주행 전체로 바꿨고,
추출기의 물리적 오류(색 → 숫자 변환)도 고쳤습니다.

1·2·3단계를 나눠 돌린 이유가 정확히 이 질문에 답하기 위해서입니다.

Q. 데이터를 고쳤는데 왜 성능이 안 올랐나?

→ 기존 변환은 틀렸지만 클래스마다 일관되게 틀려서 분류에는 쓸모가 있었습니다.

(자세한 내용은 [4차_개선작업_공유.md](#)의 “왜 다른 클래스는 잘 됐나” 참고)

고치면 우연히 잘 갈라주던 조합은 오히려 나빠질 수 있습니다.

그래도 고쳐야 하는 이유는 우연에 기대면 새 데이터에서 무너지기 때문입니다.

6. 요약

```
:: 리허설(5분) - 차종 3개 / 채널 15 확인
python 모터_학습_v3.py --data ... --out ... --group-by group_key --limit 50
```

```
:: 2단계(1h20m) - rpm 효과
python 모터_학습_v3.py --data ... --out ... --group-by group_key
```

```
:: 3단계(1h20m) - 누수 강화
python 모터_학습_v3.py --data ... --out ...
```

볼 것 우선순위:

1. [진단 2] rpm단독 F1 — 0.6 넘으면 빨간불
2. [진단 3] D→N 건수 — 3차 발표 873건과 비교
3. [진단 1] 그룹 수 — 10 미만이면 위 수치들도 흔들림

시간이 부족하면 2단계 우선. 3단계는 “얼마나 보수적으로 재느냐”의 문제라 나중에 돌려도 됩니다.